CE2 — Express.js + MySQL REST API (50.003)

Web app backend exercise building a RESTful API with Node.js, Express, TypeScript, and MySQL.

Description

Same Staff/Dept ER model as CE2 Q1, but swapped from MongoDB (document model) to MySQL (relational model). Implements `localhost:3000/dept/count` which returns the count of staff per department using a SQL `GROUP BY` query. Full MVC structure with a connection pool via `mysql2`.

Getting Started

Dependencies

- Node.js v18+
- MySQL running locally on port `3306`
- npm

Installing

Clone the repo and install dependencies:

```
git clone <repo-url>
cd ce2_q2
npm install
npm i mysql2
npm i -D @types/node @types/express typescript ts-node nodemon
```

MySQL Setup

```
CREATE DATABASE staffdir;
CREATE USER 'pichu'@'localhost' IDENTIFIED BY 'pikaP!';
GRANT ALL PRIVILEGES ON staffdir.* TO 'pichu'@'localhost';
FLUSH PRIVILEGES;
```

```
USE staffdir;

CREATE TABLE staff(
    id INTEGER PRIMARY KEY,
    name VARCHAR(255)
);
CREATE TABLE dept(
    code CHAR(2) PRIMARY KEY
);
CREATE TABLE work(
    id INTEGER,
    code CHAR(2),
    PRIMARY KEY (id),
    FOREIGN KEY (id) REFERENCES staff(id),
    FOREIGN KEY (code) REFERENCES dept(code)
);
```

Seed data:

```
INSERT INTO staff (id, name) VALUES (1, 'aaron'), (2, 'betty');
INSERT INTO dept (code) VALUES ('HR');
INSERT INTO work (id, code) VALUES (1, 'HR'), (2, 'HR');
```

Executing

Development (hot-reload):

```
./setup_and_run.sh
```

Production:

```
npx tsc && node dist/bin/www.js
```

Server runs at `http://localhost:3000`.

Available Endpoints

Method	Route	Description
GET	`/dept/count`	Count of staff per department

Testing Workflow

```
curl http://localhost:3000/dept/count
# returns: [{"count":2,"dept":"HR"}]
```

Key Difference from CE2 Q1 (MongoDB)

The count query is done entirely in SQL — no application-level looping needed:

```
SELECT COUNT(id) as count, code as dept FROM work GROUP BY code;
```

In MongoDB the equivalent required fetching all departments, then looping and calling `find()` per dept at the application tier. MySQL handles the aggregation in one query.

Known Issues & Bugs Encountered

1. `mysql2` vs `mysql` package

Cause: The lecture uses `mysql2/promise` for async/await support. The older `mysql` package does not support promises natively and will cause type errors.

Fix:

```
npm i mysql2
npm i -D @types/mysql2  # if needed
```

2. `exports default` vs `export default`

Cause: The lecture slides mix CommonJS (`module.exports`, `exports default`) and ES module (`export default`) syntax. TypeScript with `"module": "ESNext"` or `"module": "CommonJS"` will reject the wrong one.

Fix: Stick to ES module syntax throughout:

```
export default { pool, cleanup };  // db.ts
export default router;             // routes
```

---

3. `RowDataPacket[]` type for query results

Cause: `mysql2` returns query results as `RowDataPacket[]` — you need to cast properly otherwise TypeScript complains about accessing row fields.

Fix:

```
const [rows] = await db.pool.query<mysql.RowDataPacket[]>(`
    SELECT COUNT(id) as count, code as dept FROM work GROUP BY
code
`);
```

Authors

Roshan M — [50.003 Elements of Software Construction, SUTD]

Version History

- 0.1 — Initial CE2 Q2 implementation: dept/count route with MySQL GROUP BY

Acknowledgments

- DomPizzie README Template
- Express.js Docs
- mysql2 Docs
- AI for finalizing the README template